Technologies for IoT Ecosystems

Exercises on REST

Introduction

This exercise set covers the design and implementation of RESTful web services for the Internet of Things (IoT) using Python and the CherryPy framework. All services must be implemented as object-oriented classes exposed via CherryPy's MethodDispatcher, which maps HTTP verbs (GET, POST, PUT, DELETE) directly to Python methods.

Prerequisites:

- Python 3.9 or later
- CherryPy 18+ (pip3 install cherrypy)
- requests library (pip3 install requests)
- A REST client: curl, Postman, or VS Code Thunder Client

All services must be started with the following CherryPy configuration pattern:

```
if __name__ == '__main__':
    conf = {
        '/': {
            'request.dispatch': cherrypy.dispatch.MethodDispatcher(),
            'tools.sessions.on': True,
            'tools.response_headers.on': True,
            'tools.response_headers.headers': [('Content-Type',
'application/json')]
        }
    }
    cherrypy.tree.mount(MyService(), '/', conf)
    cherrypy.config.update({'server.socket_port': 9090})
    cherrypy.engine.start()
    cherrypy.engine.block()
```

Exercises

Exercise 01 — Sensor Status Checker

Implement a CherryPy REST service that accepts a sensor name as a URI segment and

returns its simulated operational status (online / offline) together with a UTC timestamp.

Endpoints	`GET` `/<sensor_name>` — returns status JSON for the named sensor
I / OExample	**Response:** `{"sensor": "temperature", "status": "online", "timestamp": "2025-01-01T12:00:00+00:00"}`
Concepts	• HTTP 404 error responses for unknown resources • UTC-aware timestamps with datetime.timezone.utc
Tips	• Maintain a set of known sensor names and return 404 if the requested name is not in the set. • Use random.choice() to simulate the online/offline status.

Exercise 02 — Threshold-Based Alert System

Build a REST service that lets clients configure min/max thresholds for named sensors and then submit readings to be checked. If a reading falls outside the configured range, an alert is generated and stored. Alerts can be queried globally or filtered by sensor.

Endpoints	`GET` `/threshold` — list all thresholds `POST` `/threshold` — set threshold {sensor, min, max} `POST` `/check` — check {sensor, value} → alert if out of range `GET` `/alerts` — list all alerts `GET` `/alerts/<sensor>` — alerts for a specific sensor

I / OExample	**Request body:**
	`{"sensor": "temperature", "value": 42.1}`
	Response:
	`{"alert": true, "details": {"sensor": "temperature", "value": 42.1, "direction": "HIGH", "timestamp": "..."}}`

Concepts	• URI-based action routing within a single CherryPy class • Cross-resource validation (check reads from threshold store) • Alert persistence and directional labelling (HIGH / LOW) • Validation: min must be strictly less than max
Tips	⚠ Route POST requests by inspecting uri[0] ('threshold' or 'check'). ⚠ Include a 'direction' field ('HIGH' or 'LOW') in the alert payload for clarity.

Exercise 03 — Multi-Room Smart Home Gateway

Design a gateway service that organises IoT devices by physical room using hierarchical URIs. Rooms are created automatically on first POST. Devices within a room are addressed by a two-segment path.

Endpoin	`GET` `/rooms` — list all rooms

ts	**GET** /rooms/<room> — list devices in a room **GET** /rooms/<room>/<device_id> — get a specific device **POST** /rooms/<room> — add a device to a room **DELETE** /rooms/<room>/<device_id> — remove a device

I / OExample	**Request body:**
	`{"name": "AirQualitySensor", "type": "co2", "value": 412}`
	Response:
	`{"message": "Device added", "device_id": "d003", "room": "bedroom"}`

Concepts	• Hierarchical URI design with nested resources • Dynamic room creation via auto-initialisation on POST • Three-level URI parsing using *uri tuple indexing • Returning structured nested JSON (room → devices)
Tips	⚠ Always check len(uri) before accessing uri[0], uri[1], uri[2]. ⚠ Auto-create the room dict if it does not exist when a device is POSTed.

Exercise 04 — Sensor Signal Simulator

Build a service that generates synthetic time-series data for named sensors (temperature, humidity, pressure, Co2, light). Support two actions: 'stream' (N random samples) and 'simulate' (N samples following a mathematical model). Available models: sine, step, sawtooth, random.

Endpoints	**GET** /stream/<sensor>?samples=N&interval=S — N random samples, optionally delayed by S seconds each **GET** /simulate/<sensor>?model=<M>&samples=N — N samples following model M
I / OExample	**Response:** `{"sensor": "temperature", "model": "sine", "samples": [{"t": 0, "value": 25.0, "unit": "°C"}, ...]}`
Concepts	• Mathematical signal generation (sine wave, step function, sawtooth) • Parametric query handling with type casting (int(), float()) • Sensor range definition and value clamping • Action-based URI routing (/stream vs /simulate)
Tips	⚠ Define SENSOR_RANGES as a class-level dict: {name: (low, high, unit)}. ⚠ For the sine model: value = mid + amp * math.sin(2 * pi * i / n).

Exercise 5 — Command Dispatcher (Actuator Control)

Build a service that models IoT actuators (LED, fan, lock, heater) and dispatches validated commands to them. Each actuator type has a set of permitted actions. The service must reject invalid action/type combinations and track the current state and last

command for each actuator.	
Endpoints	**GET** `/command` — list all actuators and their states **GET** `/command/<id>` — state of a specific actuator **POST** `/command/<id>` — send command {action, value?} to actuator

I / OExample	**Request body:**
	`{"action": "set", "value": "75%"}`
	Response:
	`{"message": "Command 'set' sent to 'fan_1'", "new_state": "75%"}`

Concepts	• Actuator state machine modelling • Per-type action validation using class-level dicts • Toggle logic (on→off, off→on) • Command audit trail (last_command with timestamp)
Tips	⚠ Define ACTUATOR_TYPES as a class-level dict mapping type names to valid action sets. ⚠ Apply the state transition in a series of if/elif blocks after validation.